

Towards Cryptography Bill of Materials Compliance

Claudio Foroncelli¹[0009-0009-4000-3760], Pietro De Matteis²[0009-0006-7684-7189], Luis Augusto Dias Knob³[0000-0001-7291-7780], Luca Piras³[0000-0002-7530-4119], Alessandro Tomasi³[0000-0002-3518-9400], and Silvio Ranise^{3,4}[0000-0001-7269-9285]

¹ Department of Control and Computer Engineering, Polytechnic University of Turin, Turin, Italy

`claudio.foroncelli@studenti.polito.it`

² Co-Innovation Lab, FBK -Dedagroup SpA, Trento, Italy

`pietro.dematteis@dedagroup.it`

³ Center for Cybersecurity, Fondazione Bruno Kessler, Trento, Italy <https://cs.fbk.eu>
`{l.diasknob,l.piras,altomasi,ranise}@fbk.eu`

⁴ Department of Mathematics, University of Trento, Via Sommarive 14, Trento, Italy

Abstract. Discovering, managing, and reporting on cryptographic assets is a critical step for the transition to quantum-safe systems and applications. Cryptography Bills of Materials (CBOMs) have been proposed as an aid to cryptographic inventory, agility, and compliance with guidelines to create more secure software and services.

Writing policies and automating compliance checks for cryptography is a valuable but complex task. We present a prototype framework for automated evaluation of cryptographic compliance extending existing CBOM tools with a policy-driven engine that classifies cryptographic assets according to customizable rules and compliance levels. Machine-readable policies enable flexible adaptation to different guidelines while supporting analysts in performing semi-automatic assessments.

The prototype is validated through experiments on both synthetic and real-world software. Results show that the system correctly identifies deprecated and disallowed primitives, producing clear compliance reports. While we highlight some difficulties common to automating compliance checks, our findings demonstrate the potential of CBOM-based approaches to enhance visibility, governance, and readiness for the post-quantum cryptography transition.

Keywords: CBOM · Compliance · Cryptography · Policy Evaluation · Post-Quantum Transition

1 Introduction

With the transition to Post-Quantum Cryptography (PQC) no longer a question of if but when [3,23,6,29], organizations must be able to discover, catalog,

and manage their cryptographic assets. This is essential to identify which cryptographic components they are using, and also to assess whether these components comply with evolving guidelines. Accurate visibility of cryptographic assets can significantly contribute to both security and regulatory compliance processes in the PQC era.

Based on the Software Bill of Materials (SBOM), CycloneDX recently developed a tool to discover, manage, and report on cryptographic assets. This object model was called Cryptography Bill of Materials (CBOM) [31], and it was explicitly designed to enable automated reasoning and compliance checking. As reported in [6], CBOMs can assist in managing and reporting the usage of cryptography when integrated into reporting and software development practices.

Generating a CBOM is a complex task, since several steps are needed to assess the software under maintenance thoroughly. First, the cryptography assets need to be mapped and added to an inventory. To generate this inventory, the application’s source code needs to be analyzed and classified, a process that is typically performed through static analysis tools, such as the CBOMKit plugin [32]. To quickly react to changes in the code, these tools are often integrated into continuous integration and deployment (CI/CD) pipelines, providing developers with immediate feedback to approve or remediate potential issues.

Notwithstanding, the simple generation of the CBOM inventory presents a great value to the administrator. It is not immediately apparent how a developer or an automated agent should take action based on these findings. To help in this process, tools like the *CBOMkit* frontend viewer provide a foundational proof-of-concept compliance check, categorizing assets into “quantum-safe” or otherwise. This feature, though useful, relies on a single hardcoded policy within the application logic, which limits its ability to validate the inventory against other standards, such as NIST SP 800-131A Rev. 3 [6].

To address this limitation, we propose a CBOM-based framework for analyzing compliance with cryptographic guidelines. The proposed approach extends the current implementation of *CBOMkit* enabling the definition of custom compliance policies and the validation of inventories against them. The definition of the policy structure ensures that each policy specifies a set of compliance levels and associated rules, which are dynamically loaded and mapped to the cryptographic assets contained in a previously generated CBOM. To demonstrate the correctness of our solution, we define custom policies derived from [6], expressed in TOML and managed independently from the application code.

To evaluate the results of our compliance analysis, we first validate the framework against a synthetic library designed to demonstrate coverage. We then apply it to the same Keycloak library used by *CBOMkit*, a widely adopted open-source identity and access management system. Based on these experiments, we identify several limitations of our approach. These limitations are further analyzed and discussed, highlighting the challenges of defining a generic compliance model across a diverse set of guidelines.

Finally, we can describe our main contributions as: (i) the definition of a policy structure, (ii) the algorithm that matches policies with CBOM, (iii) the

definition of concrete rules based on an authoritative source, and (iv) an implementation thereof. Also, our source code can be found at <https://github.com/claudioforoncelli/cbomkit>.

2 Background

2.1 Compliance and Cryptography

Cybersecurity standards agencies and groups publish recommendations and guidelines on how a service deployment should be configured to avoid insecure configurations — defining which algorithms, key lengths, and modes of operation are approved. Examples of authoritative sources include documents published by NIST [2] for the US, SOG-IS [37] for contributing European states, and individual EU member state agencies, e.g., ANSSI [1] in France or BSI [8] in Germany.

Each guideline may specify a requirement for every configurable element of a service by considering published standards, deprecated features, and known attacks. Requirement levels may be expressed using well-defined common terms [7] or custom wording. The scope of individual recommendations can be broad or specific to individual protocols, like TLS or SSH.

The existence of technical guidelines can help raise awareness and set a minimum security level. However, the lack of a standardized, machine-readable version of these documents can delay the application of recommendations and complicate the comparison between them. To manually comply with a given guideline, a system administrator must examine each requirement, speak the language, and understand the way the requirement is written, and verify that the deployment value matches the required value both by design and at runtime.

Automating compliance checks, however, is a complex endeavour, particularly for protocols that are negotiated between client and server and have gone through several versions, like TLS, but intrinsically due to the discretionary nature of the task: while some rules are mandatory, many rules are permissive and expect developers and administrators to make informed decisions. By way of example, recent work addresses the difficulty of compliance checking in TLS deployments [14]. One of the challenges involved is to map recommendations from authoritative sources into individual rules and standard requirement levels, such as [7], then defining how to combine the evaluation of individual rules on data to reach a final assessment level.

In this work, a CBOM constitutes the data, and we aim to define rules to automate compliance with the authoritative source in NIST SP 800-131A r3 [6].

2.2 The Evolution of Software Bills of Materials (SBOM)

In traditional manufacturing, a **Bill of Materials (BOM)** refers to a comprehensive list of raw materials, components, and assemblies required to construct a product. This concept has been adapted to the software domain to address the growing complexity of software supply chains and the increasing emphasis on cybersecurity.

A **Software Bill of Materials (SBOM)** is an inventory of all components within a software product, with particular attention to third-party and open-source software (OSS) libraries. SBOMs provide visibility into nested dependencies, licensing information, and known vulnerabilities, such as those cataloged by the *Common Weakness Enumeration (CWE)* and *Common Vulnerabilities and Exposures (CVE)* systems.

The importance of SBOMs was boosted by major cybersecurity incidents, notably the *SolarWinds breach (2020)* [24] and the *Log4j vulnerability (2021)* [25]. In response, the U.S. government issued **Executive Order 14028** [40] in 2021, mandating the use of SBOMs for software used by federal agencies. This directive catalyzed global recognition of SBOMs as a foundational element in enhancing software supply chain security. The relevance of SBOMs is further reflected in international regulatory frameworks, such as the European Union’s **Cyber Resilience Act (CRA)** [13], or guidances that present a shared vision of the importance of SBOMs in the global community [10].

SBOMs enhance security by enabling rapid identification and mitigation of vulnerabilities in third-party components. They support risk management by providing a comprehensive view of software composition, allowing organizations to assess and control risks associated with untrusted dependencies. SBOMs also facilitate license compliance by clarifying the legal status of included libraries, and contribute to regulatory compliance by supporting adherence to standards, policies, and legal requirements.

The *NTIA Software Transparency Working Group on Standards and Formats* [27] considers two of the major open source Standard formats for SBOMs:

- **Software Package Data Exchange (SPDX)**: Developed by the Linux Foundation, SPDX is a machine-readable format standardized as ISO/IEC 5962:2021 [39].
- **CycloneDX (CDX)**: Originating from the OWASP community, CycloneDX is also a machine-readable format and has been standardized as ECMA-424 [30].

Both SPDX 3.0 and CycloneDX 1.6 are evolving to cover emerging domains and technologies. This evolution has led to the conceptual expansion of SBOMs into specialized variants, such as the **AI Bill of Materials (AIBOM)**, the **Cryptography Bill of Materials (CBOM)**, the **Operations Bill of Materials (OBOM)**, and the **Hardware Bill of Materials (HBOM)**. These extensions aim to provide comprehensive transparency across diverse technological ecosystems, further reinforcing the role of BOMs in securing digital infrastructure.

2.3 Cryptography Bill of Materials (CBOM)

Cryptography Bill of Materials (CBOM) is a structured, machine-readable inventory that enumerates all cryptographic components present in a software system. Created as an extension of the Software Bill of Materials (SBOM), the CBOM focuses specifically on cryptographic assets, including algorithms, keys, certificates, and protocols. By providing an authoritative record of these elements, the

CBOM becomes a valuable resource for security audits, compliance verification, and cryptographic risk assessments.

By representing cryptographic information in a uniform format, CBOMs enable automation, interoperability, and traceability across tools and processes. They also offer visibility into the cryptographic posture of a system, allowing for the detection of deprecated algorithms, weak key lengths, or misconfigured protocols. At the same time, it provides input for compliance checks against guidelines and standards from bodies such as NIST, ENISA, and national cybersecurity agencies.

Although documents from the Office of Management and Budget (OMB) [41], and NIST [26] focused on the preparation for a post-quantum cryptography (PQC) migration, do not explicitly create the definition of a specific Bill of Materials for cryptographic assets. These guidelines highlighted the importance of establishing deprecation and disallowance timelines, as well as prioritizing the inventory of cryptographic systems. Therefore, it was natural that this inventory would eventually be formalized and its structure well-defined for an external organization or company.

NIST SP 1800-38B [28] uses a profile of *CycloneDX* [31], an SBOM specification maintained by the OWASP Foundation, to generate a cryptography inventory. CycloneDX was initially designed for software component inventories. However, its extensibility allows for domain-specific extensions. The CBOM profile originally defined at IBM contains cryptography-specific component types (e.g., `cryptographic-asset`) and properties (e.g., `cryptoProperties`, `protocolProperties`, `tlsCipherSuites`); this aligns cryptographic asset inventories with a widely supported standard while tailoring them to meet the needs of PQC migration.

2.4 CBOM structure

CycloneDX CBOM is an SBOM profile that defines an object model to describe cryptographic assets and their dependencies. The CBOM object model is a standardized representation of cryptographic assets. Figure 1 shows a graphical definition of the asset types and subtypes available. More specifically, four main asset types are defined:

Algorithm assets can specify fields such as `primitive`, `mode` (mode of operation), `padding`, or `cryptoFunctions`.
Certificate assets expose fields like `issuerName`, `subjectName`, `notValidAfter`, and `signatureAlgorithmRef`, but do not have attributes such as `mode` or `primitive`.
Protocol assets include `type`, `version`, and potentially a list of `cipherSuites`.
Related crypto material assets include fundamental and ancillary building blocks of different kinds, such as `key`, `salt`, `token`, `credential`, etc.

The granular nature of the object model lends itself to the definition of fine-grained policies. For instance, a policy for a signature algorithm as in Listing 1.1 can be written as separate rules for the functions `sign` and `verify`.

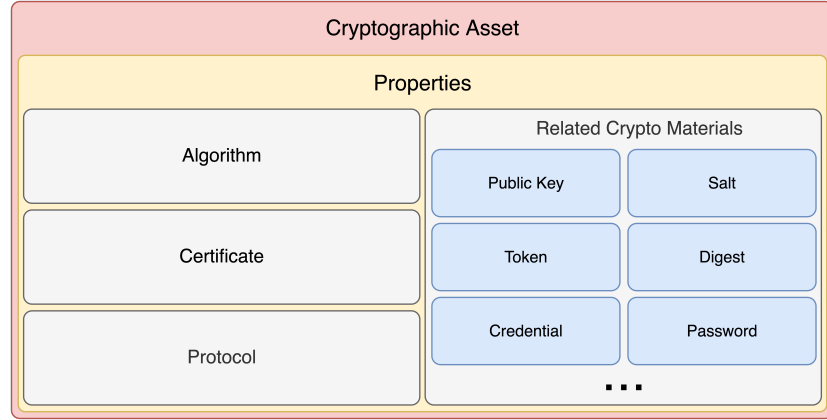


Fig. 1. CBOM Object Model

2.5 CBOM Generation

Several solutions have been published to generate CBOM in the CI/CD pipeline: *CBOMkit* [32] via SonarQube rules [33], *cdxgen* [11], and *cbom-action* [15] via GitHub CodeQL [16]. Other sources have been demonstrated, such as *CryptoMon* for network traffic [35].

cdxgen can generate CBOMs, but its primary focus remains on SBOMs, and its cryptographic modeling is still basic. *cbom-action* is centered on generation alone, without providing a viewer, compliance analysis, or management features.

CBOMkit stands out as the only solution capable of analyzing source code directly, not just compiled binaries. It offers a viewer for interactive exploration and a compliance engine to evaluate CBOMs against policies.

3 A CBOM-Based Framework for Analysing Compliance with Standards and Guidelines

Each rule defines one or more constraints on the properties of cryptographic assets. When an asset matches all of the constraints defined in a rule, that rule’s compliance level is assigned. Compliance levels are asset-specific; the compliance tool translates fine-grained, asset-specific policies into an assessment level — a single global result.

3.1 Compliance levels

Compliance levels are derived from the guideline with which compliance is to be checked. For [6], a distinction is made between applying new protection –

e.g., encrypting signing – and processing already protected information, e.g., decrypting or verifying. Further details on this are discussed in Section 4.3.

The four compliance levels explicitly defined in the authoritative source are:

- 2: Acceptable.** The algorithm, key length/strength, parameter set, or scheme MAY be used.
- 3: Legacy Use.** MAY be used for processing already protected information when applying new protection with the same algorithm would be disallowed.
- 4: Deprecated.** carries some risk but MAY be used following a risk assessment.
- 5: Disallowed.** MUST NOT be used to apply new protection.

Note that there is a base level defined by *CBOMkit* as “1: Unknown”. This is an interesting corner case because on the one hand, guidelines do not explicitly specify every possible value for every constraint, but on the other, they do sometimes prohibit anything not explicitly specified. For example, in NIST TLS guidelines [22] Section 3.3.1, “Cipher suites that do not appear in this [document] **shall not** be used”. The full public list of cipher suites is maintained by IANA [17], so a compliance policy may be written as either assigning a higher compliance severity level to unknown findings, or explicitly assigning a Disallowed level to all possible values not explicitly listed in the authoritative source.

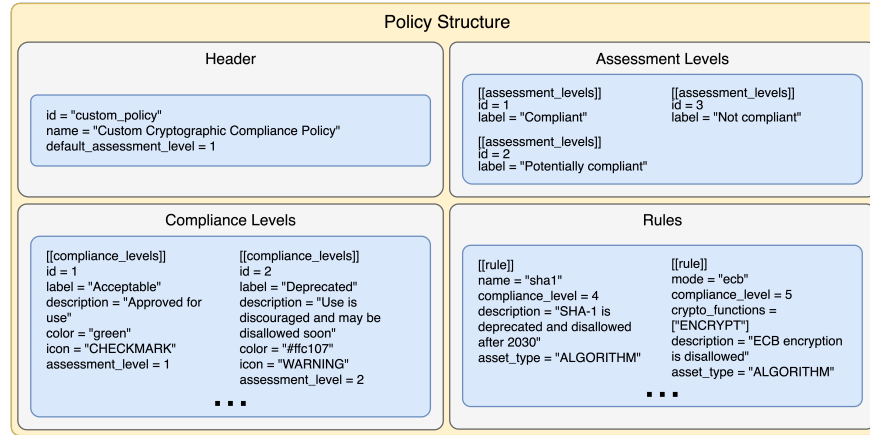


Fig. 2. Compliance Creation Structure

3.2 Policies and rule matching

A policy is a container for a set of rules, with a header with metadata and a default assessment level to be used if no matching rules are found. This is a common pattern for e.g., access control policy languages like XACML.

Each rule contains:

- **name**: An optional string used to match the name of the cryptographic asset.
- **compliance_level**: The compliance level to assign if the rule matches.
- **description**: A human-readable explanation of the rule’s purpose.
- **asset_type**: The type of asset this rule applies to (see Section 2.4); determines the set of attributes to be matched with the properties of cryptographic assets in a CBOM.
- One or more matching fields from the asset’s property structure.

See Section A for examples of rules.

During policy evaluation, an asset in a given CBOM matches a rule in a policy if *all* non-null fields specified in the rule exactly match the corresponding properties of the asset.

In cases where multiple rules apply to the same asset, *CBOMkit* resolves conflicts by giving precedence to the rule with the highest *specificity* i.e., the rule that defines the greatest number of matching constraints on the asset’s properties. If two rules have equal specificity, the one with the higher compliance level i.e., severity is chosen. After per-asset evaluations are completed, the engine aggregates these individual compliance levels into an overall assessment that reflects the system’s conformity with the policy’s defined levels. The assessment level is chosen among the ones defined in the policy (see Section 3.3).

To support parameter range conditions – e.g., key length $l \geq 128$, or $128 \leq k < 256$ – in addition to simple equality checks, we also modify the compliance engine, which previously could only verify whether an asset’s parameter exactly matched the value defined in a rule.

3.3 Assessment levels

Assessment levels are defined by the engine as one of three possible values:

- 1: Compliant.** All assets conform to the defined mandatory rules.
- 2: Potentially not compliant.** Some assets fall under discretionary rules recommending against — deprecated or unknown; all other assets are compliant with mandatory rules.
- 3: Non-compliant.** One or more assets use algorithms or configurations explicitly disallowed by the policy.

Each assessment level may be referenced by multiple compliance levels, allowing asset-specific classifications to be mapped into a single global indicator. The aggregation process thus translates specific per-asset findings into a unified compliance status, highlighting the most severe level of non-conformance detected within the analyzed system.

3.4 Policy Configuration Language

TOML (Tom’s Obvious, Minimal Language) [34] is a configuration file format designed to be minimal, easy to read, and semantically unambiguous. It is designed

to map to key-value pairs and support hierarchical data, making it suitable for a broad range of configuration use cases.

TOML was chosen to represent our policies to ensure that policies can be easily authored and maintained even by users without programming expertise. Its clear, human-readable syntax provides the flexibility required to accommodate the structural and semantic needs of the extended software. CSV or equivalent spreadsheet formats were discarded due to the redundancy they would introduce, as many rows would need to be repeated with minimal variation; and JSON was excluded on the basis that its syntax is less intuitive for non-technical users.

4 Evaluation

To evaluate the correctness of the evaluation logic and custom policy, we apply the extended CBOMkit compliance engine to two CBOMs: a synthetic test CBOM designed to include representative assets of every type and compliance level (Section 4.1), and the CBOM generated for Keycloak [16, 15], a widely used open-source identity and access management solution (Section 4.2).

4.1 Test CBOM

Figure 3 shows the frontend viewer result of the compliance analysis for the test CBOM. In addition to the *Deprecated* result corresponding to an instance of the SHA-1 hash function (Figure 4), the analysis also identified several other cryptographic primitives of interest. Instances of HMAC-SHA256 were detected and correctly categorized as *Acceptable*. The SHA-224 function was labeled as *Deprecated*, consistent with its phase-out under the policy. Furthermore, the AES128-ECB-PKCS5 construct was classified as *Disallowed*, as ECB mode is explicitly prohibited for encryption operations. Conversely, AES128-CBC-PKCS5 was assessed as *Acceptable*, since both the AES algorithm and the CBC mode of operation are approved for use under the same standard.

Due to the presence of various *Disallowed* assets, the overall assessment level of the CBOM was successfully deemed *Not Compliant*.

4.2 Keycloak CBOM

Figure 5 shows the frontend viewer result of the compliance analysis for the Keycloak CBOM. Details of the *Deprecated* result, corresponding to an instance of the SHA-1 hash function, are shown in Figure 6.

In addition to hash function findings, the analysis also identified instances of elliptic-curve-based key generation algorithms using the P-384 and P-521 curves. According to the recommendations outlined in NIST SP 800-186 [9], these curves are among the recommended elliptic curves for federal use. Consequently, the compliance engine classified these instances as *Acceptable*, confirming that the ECDSA key generation mechanisms employed in Keycloak conform to current NIST guidance for approved elliptic-curve parameters.

The SHA-1 finding is non-trivial to assess; see Section 4.3.

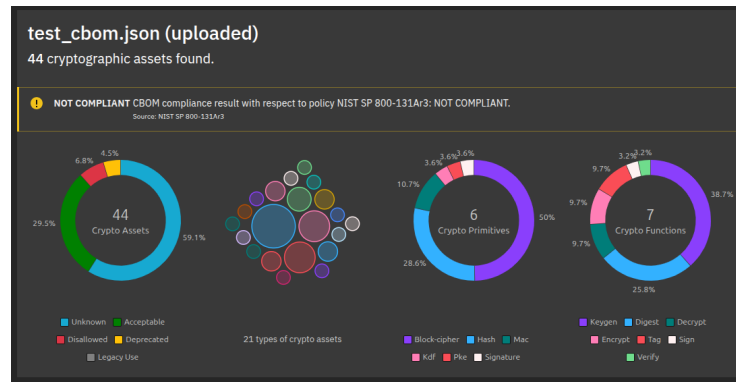


Fig. 3. Test CBOM stat screen.

List of all assets Download CBOM

Cryptographic asset	Type	Primitive	Location
AES128-ECB-PKCS5	Algorithm	Block Cipher	SymmEncECB.java:19
SECRET-KEY	Related Crypto Material	Unspecified	NistRulesDemo.java:37
SHA1	Algorithm	Hash Function	NistRulesDemo.java:21
SECRET-KEY	Related Crypto Material	Unspecified	SymmEncECB.java:11
3DES	Algorithm	Block Cipher	NistRulesDemo.java:27
3DES	Algorithm	Block Cipher	NistRulesDemo.java:30

Fig. 4. Test CBOM asset list.

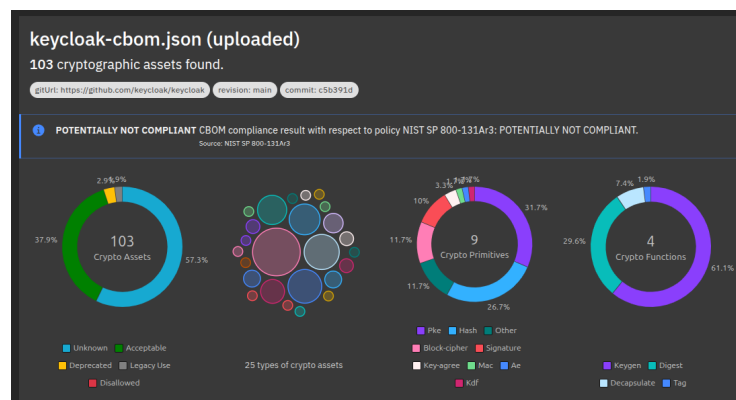


Fig. 5. Keycloak CBOM stat screen.

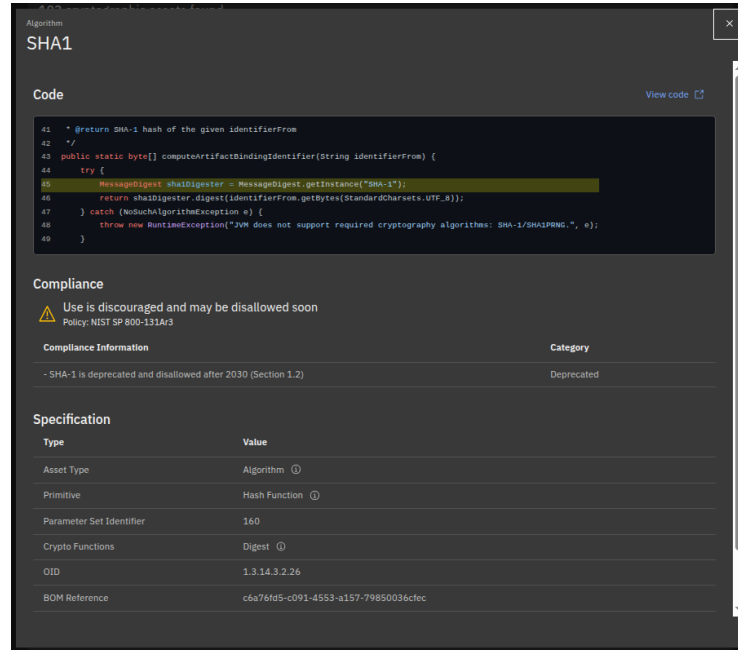


Fig. 6. Keycloak CBOM SHA-1 finding.

4.3 Benefits and limitations

There are at least three criteria on which the framework here proposed could be assessed: whether it is possible and easy enough to write policies based on guidelines; how correct and complete the resulting assessment is; and how useful the assessment is to developers. All of these likely require expert assessment and trial, which we do not address here. Alternatively, automating the assessment of an automated assessment tool would be an interesting research question.

To the first and second point we provide remarks on the difficulties encountered in writing Elliptic Curve policies and in evaluating SHA-1 results, below. The effectiveness of TOML should be assessed in practice by others for an unbiased view; while it should not require IT-technical expertise, reading the recommendations and writing the rules still requires a certain degree of familiarity with cryptographic algorithms.

To the third point, one of the main limitations of *CBOMkit* is that currently very few libraries are available for analysis: python cryptography, JCA, and bouncycastle. This also limits the applications we can consider to evaluate our framework.

Evaluating policy writing for elliptic curve cryptography.

Limitations of CBOMs based on CycloneDX 1.6. CBOMs produced using CycloneDX 1.6 exhibited several limitations that affected the accuracy and reliability of automated cryptographic compliance checking. Most notably, elliptic curves were represented using free-form text strings with no canonical vocabulary, meaning that a single curve could appear under multiple aliases—such as “B-233” and “Sect233r1”—depending on the library or naming convention in use. Determining the corresponding security strength therefore required manual cross-referencing with external sources such as [9,36]. Similar inconsistencies arose for primitives and algorithm families, which lacked a unified schema and had to be inferred from implementation details (e.g. Java class names), leading to incomplete or inconsistent detection. While integer-based schemes such as RSA could typically be evaluated from a single parameter, elliptic-curve assets often required custom mapping logic, and some curves listed in [9] were not detectable (see Table 2). These limitations made it difficult to express policies in a portable way or to ensure consistent and reproducible compliance evaluations.

Enhancements introduced with CycloneDX 1.7. The release of CycloneDX 1.7 is a substantial improvement through the introduction of a structured *Cryptography Registry* providing unique identifiers for elliptic curves, algorithm families, primitives, and parameter sets. This registry replaces the earlier reliance on ad hoc text labels and aligns nomenclature across NIST, SEC 2, Brainpool, and commonly used cryptographic implementations. Policies can now reference stable identifiers rather than implementation-specific strings, eliminating the need for custom mappings and increasing interoperability between CBOM producers and consumers. The registry also expands coverage and accuracy: curves or algorithms that previously appeared under ambiguous or deprecated aliases can now be identified consistently. Certain policy constraints remain inherently contextual—such as the allowance of Secp256k1 solely “for blockchain applications” in [9]—and thus require additional reasoning beyond CBOM metadata. Nevertheless, CycloneDX 1.7 provides a significantly stronger and more structured foundation for automated cryptographic evaluation, addressing many of the limitations observed in CBOMs based on version 1.6.

Evaluating SHA-1 compliance. SHA-1 is a good example of the limitations of automated compliance testing. In revision 2 of SP 800-131A [5], currently still valid while revision 3 is in draft stage, SHA-1 is (our emphasis):

- **disallowed** for digital signature *generation* **except where specifically allowed** by NIST protocol-specific guidance;
- **allowed for legacy use** when used for digital signature *verification*;
- **acceptable** for non-digital signature applications that do not require collision resistance.

A concrete example of protocol-specific guidance explicitly allowing SHA-1 for signature generation are NIST TLS guidelines [22], where it is allowed for TLS versions 1.0 and 1.1 ServerKeyExchange (section 3.3.1.1.1 footnote 21) and CertificateVerify (section 3.5 footnote 26) messages.

In the new draft revision 3, both exception and reference to collision resistance have been dropped; instead, the notion of “applying cryptographic protection”. An excerpt of the two versions is shown side by side in Table 1.

Table 1. SHA-1 compliance in SP 800-131A Rev. 2 [5], Table 8 and Rev. 3 [6], Table 12.

Rev. 2		Rev. 3	
Use	Status	Use	Status
Digital signature generation	Disallowed, except where specifically allowed by NIST protocol-specific guidance.	Digital signature generation	Disallowed
Digital signature verification	Legacy use	Digital signature verification	Legacy use
Non-digital signature applications	Acceptable	Applying protection for non-digital signature applications	Deprecated through 2030 Disallowed after 2030
		Processing already protected information using non-digital signature applications	Acceptable through 2030 Legacy use after 2030

This presents the following challenges: in terms of compliance with [22], categorizing a finding of SHA-1 in a CBOM requires (a) checking whether it is being used for signature generation or verification; (b) checking which protocol it is being used in, and hence against other applicable guidelines published by NIST; and (c) making a judgement call as to whether the application does or does not require collision resistance, which requires a certain degree of security expertise and an evaluation of the context in which the primitive is being used.

(a) *Checking against algorithm function* can be addressed with CBOM by specifying a different compliance level for the two functions in relevant signature algorithms. This can be considered a success for CBOM. Rule definition is partially limited by the dictionary of algorithms – at the time of writing, the CycloneDX specification follows the JSON Signature Format (JSF) standard, which in turn

refers⁵ to JWA [19] and JOSE [21]. This requires going beyond the github and RFC documents to the IANA registry [18], where SHA-1 algorithms (RS1, HS1) are listed – and explicitly prohibited.

(b) *Checking against other guidelines on use in a specific protocol* may be addressed by setting a default base value of Disallowed for the primitive in this guideline, then resolving conflicts with other guidelines. The resolution must be able to handle precedence in a way that is counter-intuitive to security, in the sense that the more permissive compliance level takes precedence if the guideline is protocol-specific. This likely requires further work to determine how to handle policy conflicts in general as it cannot be addressed by specifying a single guideline policy in isolation. Note that, even when an exception is not explicitly specified – such as its removal between revision 2 and 3 of [6] – conflicts may still arise and need to be resolved somehow; in other words, even though revision 3 does not specify that exceptions are acceptable, the permission to use SHA-1 in specific cases is not erased from [22].

(c) *Expert discretion* may be unavoidable in some cases. For instance, deciding whether an application of a hash function relies on the security property of collision resistance can hardly be automated. Collision resistance is one of several security properties of the primitive, as pointed out in the referenced publication [12]. Arguments could then be made about whether chosen prefix collisions might be undesirable in the specific application, such as defeating code signing or crafting a rogue CA certificate as demonstrated for MD5 [38], and to what extent that applies for a hash function with the same construction such as SHA-1 [20]. Fortunately in revision 3 the phrase “applying cryptographic protection” is defined as (our emphasis): “Depending on the algorithm, to encrypt or sign data, *generate a hash function* or message authentication code (MAC), or establish keys, including wrapping and deriving keys” [6, Appendix D]. While this clarifies that generating a new message digest is deprecated, there is no intrinsic distinction in calling a hash function to compute a digest for a new message or for the purpose of comparing the computed digest with an existing one, so an evaluation of the context is still necessary.

5 Conclusions

CBOMkit was extended with a structured, machine-readable policy format in TOML, capturing metadata, compliance levels, assessment levels, and rules. A Java evaluation framework was built, including a policy parser and an evaluation engine, which together allow both built-in and user-defined policies to be applied. Validation on a test CBOM and the CBOM of Keycloak, a real-world identity and access management platform, showed that the system correctly flags deprecated or disallowed algorithms and propagates these findings into a system-level compliance posture through assessment levels.

⁵ <https://github.com/IBM/CBOM/blob/main/jsf-0.82.schema.json>

While this establishes a solid foundation for cryptographic compliance evaluation, several opportunities for future work remain.

Future work. Current analysis depends on CBOMs from static analyzers with limited language and library coverage, mainly Java and Python. Many cryptographic usages, especially in custom routines or niche libraries, may remain undetected by static approaches. Automating mapping from libraries to CBOM element detection at least in part would be highly beneficial, as it would both extend coverage to new libraries as well as ease the process of keeping support apace with library development – especially relevant given the expected integration of new PQC standard algorithms and hybrids. In addition to new algorithms being standardized and implemented, the computational assumptions of algorithms currently considered quantum resistant may turn out to be flawed, as in the case of SIKE; new algebraic attacks could reduce security, leading to frequent changes in security guidelines. Absent a common machine-readable format used by guideline-publishing authorities, automating the extraction of policies from guidelines would reduce the burden of keeping up with updates to technical documents.

Reference documents such as guidelines, standards and regulations may overlap and disagree on compliance levels of specific items. When dealing with multiple reference documents, analysts need guidance and tools able to identify such overlaps, prioritise them, and receive support towards fulfilling related requirements. Future work should address policy combining algorithms for multiple reference documents, and integrate support for developers as appropriate. Evaluation through interviews with developers and users on the usefulness of CBOM and compliance tool would also be desirable.

Disclosure of Interests. The authors have no competing interests to declare.

References

1. ANSSI: Mécanismes cryptographiques: Règles et recommandations. <https://cyber.gouv.fr/publications/mecanismes-cryptographiques> (Jan 2020)
2. Barker, E.: Guideline for using cryptographic standards in the federal government: Cryptographic mechanisms. <https://csrc.nist.gov/pubs/sp/800/175/b/r1/final> (Mar 2020). <https://doi.org/10.6028/NIST.SP.800-175Br1>, NIST SP 800-175B Revision 1
3. Barker, E., Chen, L., Cooper, D., Moody, D., Regenscheid, A., Souppaya, M., Newhouse, W., Housley, R., Turner, S., Barker, W., Scarfone, K.: Considerations for achieving crypto agility: Strategies and practices. <https://csrc.nist.gov/pubs/cswp/39/considerations-for-achieving-cryptographic-agility/2pd> (Jul 2025). <https://doi.org/10.6028/NIST.CSWP.39.2pd>, NIST CSWP 39 (2pd)
4. Barker, E., Chen, L., Roginsky, A., Vassilev, A., Davis, R.: Recommendation for pair-wise key-establishment schemes using discrete logarithm cryptography. <https://csrc.nist.gov/pubs/sp/800/56/a/r3/final> (Apr 2018). <https://doi.org/10.6028/NIST.SP.800-56Ar3>, nIST Special Publication 800-56A Revision 3

5. Barker, E., Roginsky, A.: Transitioning the use of cryptographic algorithms and key lengths. <https://csrc.nist.gov/pubs/sp/800/131/a/r2/final> (Mar 2019). <https://doi.org/10.6028/NIST.SP.800-131Ar2>, NIST SP 800-131A Rev. 2
6. Barker, E., Roginsky, A.: Transitioning the use of cryptographic algorithms and key lengths. <https://csrc.nist.gov/pubs/sp/800/131/a/r3/ipd> (Oct 2024). <https://doi.org/10.6028/NIST.SP.800-131Ar3.ipd>, NIST SP 800-131Ar3 ipd
7. Bradner, S.O.: Key words for use in RFCs to indicate requirement levels (RFC 2119). <https://www.rfc-editor.org/info/rfc2119> (Mar 1997). <https://doi.org/10.17487/RFC2119>
8. BSI: Technical Guideline TR-02102 Cryptographic Mechanisms. https://www.bsi.bund.de/EN/Themen/Unternehmen-und-Organisationen/Standards-und-Zertifizierung/Technische-Richtlinien/TR-nach-Thema-sortiert/tr02102/tr02102_node.html (January 2025)
9. Chen, L., Moody, D., Regenscheid, A., Robinson, A., Randall, K.: Recommendations for discrete logarithm-based cryptography: Elliptic curve domain parameters. <https://csrc.nist.gov/pubs/sp/800/186/final> (Feb 2023). <https://doi.org/10.6028/NIST.SP.800-186>
10. CISA and the National Security Agency (NSA): A Shared Vision of Software Bill of Materials (SBOM) for Cybersecurity. <https://www.cisa.gov/resources-tools/resources/shared-vision-software-bill-materials-sbom-cybersecurity>, accessed: 2025-10-07
11. CycloneDX Project: **cdxgen**: Cyclonedx generator. <https://github.com/CycloneDX/cdxgen>
12. Dang, Q.: Recommendation for applications using approved hash algorithms. <https://csrc.nist.gov/pubs/sp/800/107/r1/final> (Aug 2012). <https://doi.org/10.6028/NIST.SP.800-107r1>, NIST SP 800-107 Revision 1
13. EU: Cyber Resilience Act (CRA). <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX%3A32024R2847> (October 2024)
14. Germenia, R., Manfredi, S., Rizzi, M., Sciarretta, G., Tomasi, A., Ranise, S.: Automating compliance for improving TLS security postures: An assessment of public administration endpoints. In: SECRIPT. pp. 450–458. SciTePress (2024). <https://doi.org/10.5220/0012764700003767>
15. GitHub Advanced Security: CBOM action: Generate cryptography bill of materials with CodeQL. <https://github.com/advanced-security/cbom-action>
16. GitHub Security Lab: Addressing post-quantum cryptography with CodeQL. <https://github.blog/security/vulnerability-research/addressing-post-quantum-cryptography-with-codeql/> (Dec 2023)
17. IANA: Transport Layer Security (TLS) parameters. <https://www.iana.org/assignments/tls-parameters/> (2005)
18. IANA: JSON Object Signing and Encryption (JOSE). <https://www.iana.org/assignments/jose/jose.xhtml> (Aug 2025)
19. Jones, M.B.: JSON Web Algorithms (JWA). RFC 7518 (May 2015). <https://doi.org/10.17487/RFC7518>, <https://www.rfc-editor.org/info/rfc7518>
20. Leurent, G., Peyrin, T.: SHA-1 is a shambles: First Chosen-Prefix collision on SHA-1 and application to the PGP web of trust. In: 29th USENIX Security Symposium (USENIX Security 20). pp. 1839–1856. USENIX Association (Aug 2020), <https://www.usenix.org/conference/usenixsecurity20/presentation/leurent>
21. Liusvaara, I.: CFRG Elliptic Curve Diffie-Hellman (ECDH) and Signatures in JSON Object Signing and Encryption (JOSE). RFC 8037 (Jan 2017). <https://doi.org/10.17487/RFC8037>, <https://www.rfc-editor.org/info/rfc8037>

22. McKay, K.A., Cooper, D.A.: Guidelines for the selection, configuration, and use of transport layer security (TLS) implementations. <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-52r2.pdf> (Aug 2019). <https://doi.org/10.6028/NIST.SP.800-52r2>
23. Moody, D., Perlner, R., Regenscheid, A., Robinson, A., Cooper, D.: Transition to post-quantum cryptography standards. <https://csrc.nist.gov/pubs/ir/8547/ipd> (Nov 2024). <https://doi.org/10.6028/NIST.IR.8547.ipd>, NIST IR 8547 (IPD)
24. National Institute of Standards and Technology (NIST): National Vulnerability Database - CVE-2020-10148 (SolarWinds Orion API Vulnerability) . <https://nvd.nist.gov/vuln/detail/CVE-2020-10148>, accessed: 2025-10-08
25. National Institute of Standards and Technology (NIST): National Vulnerability Database - CVE-2021-44228 (Apache Log4j2 Vulnerability). <https://nvd.nist.gov/vuln/detail/cve-2021-44228>, accessed: 2025-10-07
26. National Institute of Standards and Technology (NIST): NIST interagency report (NIST IR) 8547: Cryptographic asset management. Tech. Rep. NIST IR 8547 (Initial Public Draft), National Institute of Standards and Technology (NIST) (Nov 2024). <https://doi.org/10.6028/NIST.IR.8547.ipd>, <https://doi.org/10.6028/NIST.IR.8547.ipd>
27. National Telecommunications and Information Administration (NTIA): The minimum elements for a software bill of materials (sbom). https://www.ntia.doc.gov/files/ntia/publications/sbom_minimum_elements_report.pdf, accessed: 2025-10-07
28. Newhouse, W., Souppaya, M., Barker, W., Brown, C.: Migration to post-quantum cryptography: Quantum readiness – cryptographic discovery (preliminary draft). Special Publication 1800-38B, National Institute of Standards and Technology (NCCoE), Gaithersburg, MD (Dec 2023), <https://www.nccoe.nist.gov/crypto-agility-considerations-migrating-post-quantum-cryptographic-algorithms>, preliminary Draft; Released for public comment December 19, 2023
29. Newhouse, W., Souppaya, M., Barker, W., Brown, C., Kampanakis, P., Manzano, M., McGrew, D., Dames, A., Soukharev, V., Lafrance, P., Hu, A., Hook, D., Garcia, R., Evgeny, G., Kim, E., Lee, C.: Migration to post-quantum cryptography: Quantum readiness: Cryptographic discovery. <https://www.nccoe.nist.gov/crypto-agility-considerations-migrating-post-quantum-cryptographic-algorithms> (dec 2023), NIST SP 1800-38B (ipd)
30. OWASP Foundation: Cyclonedx (CDX). <https://cyclonedx.org/>, accessed: 2025-10-07
31. OWASP Foundation: Authoritative guide to cryptography bill of materials (CBOM). https://cyclonedx.org/guides/OWASP_CycloneDX-Authoritative-Guide-to-CBOM-en.pdf (Apr 2024)
32. Post-Quantum Cryptography Alliance: CBOMkit: An open-source toolkit for cryptography bills of materials. <https://github.com/PQCA/cbomkit>
33. Post-Quantum Cryptography Alliance: Sonar cryptography: Detection rule structure. https://github.com/PQCA/sonar-cryptography/blob/main/docs/DETECTION_RULE_STRUCTURE.md
34. Preston-Werner, T.: TOML: Tom’s Obvious, Minimal Language. <https://github.com/toml-lang/toml> (Jan 2021), accessed: 2025-08-29
35. Santander Security Research: CryptoMon: Network cryptography monitor - using ebpf, written in python. <https://github.com/Santandersecurityresearch/CryptoMon>
36. SEC2G: Recommended elliptic curve domain parameters. <https://www.secg.org/sec2-v2.pdf> (Jan 2010), standards for Efficient Cryptography (SEC) 2, Version 2.0

37. SOG-IS: SOG-IS crypto evaluation scheme: Agreed cryptographic mechanisms. https://www.sogis.eu/uk/supporting_doc_en.html (Feb 2023), version 1.3
38. Stevens, M.M., Lenstra, A.K., de Weger, B.M.M.: Chosen-prefix collisions for md5 and applications. *International Journal of Applied Cryptography* **2**(4), 322–359 (2012). <https://doi.org/10.1504/IJACT.2012.048084>, <https://marc-stevens.nl/research/hashclash/>
39. The Linux Foundation: The System Package Data Exchange (SPDX). <https://spdx.dev/>, accessed: 2025-10-07
40. US Government: President’s Executive Order (EO) 14028 on Improving the Nation’s Cybersecurity. <https://www.federalregister.gov/documents/2021/05/17/2021-10460/improving-the-nations-cybersecurity> (May 2021), accessed: 2025-10-07
41. Young, S.D.: Migrating to post-quantum cryptography. Memorandum M-23-02, Executive Office of the President, Office of Management and Budget (OMB), Washington, DC (Nov 2022), <https://www.whitehouse.gov/wp-content/uploads/2022/11/M-23-02-M-Memo-on-Migrating-to-Post-Quantum-Cryptography.pdf>

Appendix

A Example TOML snippet – Rules for algorithms

This example defines two rules targeting cryptographic algorithms: one deprecating SHA-1, and one disallowing ECB mode for encryption.

```
[[rule]]
name = "sha1"
compliance_level = 4
description = "SHA-1 is deprecated and disallowed after 2030"
asset_type = "ALGORITHM"

[[rule]]
mode = "ecb"
compliance_level = 5
crypto_functions = ["ENCRYPT"]
description = "ECB encryption is disallowed"
asset_type = "ALGORITHM"
```

B Example CBOM listing – signature algorithm

Listing 1.1. CBOM example of signature. Source: [31].

```
{
  "name": "SHA512withRSA",
  "type": "cryptographic-asset",
  "cryptoProperties": {
    "assetType": "algorithm",
    "algorithmProperties": {
      "primitive": "signature",
```

```

    "parameterSetIdentifier": "512",
    "executionEnvironment": "software-plain-ram",
    "implementationPlatform": "x86_64",
    "certificationLevel": [
      "none"
    ],
    "cryptoFunctions": [
      "sign",
      "verify"
    ],
    "nistQuantumSecurityLevel": 0
  },
  "oid": "1.2.840.113549.1.1.13"
}

```

C Elliptic Curve mapping

Table 2. Elliptic Curve mapping between security strength, identifying strings, and recommendation for signature and key establishment (KE) [4] from [9], [36], and *CBOMkit* in CycloneDX 1.6. Blank cells indicate no corresponding string is present in the source.

Security bits	SP 800-186 string	<i>CBOMkit</i> string	SP 800-186 recommendation
80		Sect163k1	
80		Sect163r2	
96		Secp192r1	
112	P-224	Secp224r1	Allowed for ECDSA, EC KE
112	K-233	Sect233k1	Deprecated
112	B-233	Sect233r1	Deprecated
112	brainpoolP224r1		Allowed for ECDSA, EC KE
128	P-256	Secp256r1	Allowed for ECDSA, EC KE
128	secp256k1	Secp256k1	Allowed to be used for blockchain-related applications
128	W-25519		Disallowed for ECDSA or EdDSA
128	Curve25519	Curve25519	Disallowed for ECDSA or EdDSA
128	Edwards25519	Edwards25519	Allowed for EdDSA
128	K-283	Sect283k1	Deprecated
128	B-283	Sect283r1	Deprecated
128	brainpoolP256r1	Brainpoolp256r1	Allowed for ECDSA, EC KE
160	brainpoolP320r1		Allowed for ECDSA, EC KE
194	P-384	Secp384r1	Allowed for ECDSA, EC KE
194	K-409	Sect409k1	Deprecated
194	B-409	Sect409r1	Deprecated
194	brainpoolP384r1	Brainpoolp384r1	Allowed for ECDSA, EC KE
224	W-448		Disallowed for ECDSA or EdDSA
224	Curve448	Curve448	Disallowed for ECDSA or EdDSA
224	Edwards448	Edwards448	Allowed for EdDSA
224	E448		Disallowed for ECDSA or EdDSA
256	brainpoolP512r1	Brainpoolp512r1	Allowed for ECDSA, EC KE
256	P-521	Secp521r1	Allowed for ECDSA, EC KE
256	K-571	Sect571k1	Deprecated
256	B-571	Sect571r1	Deprecated